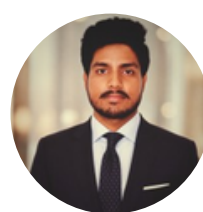
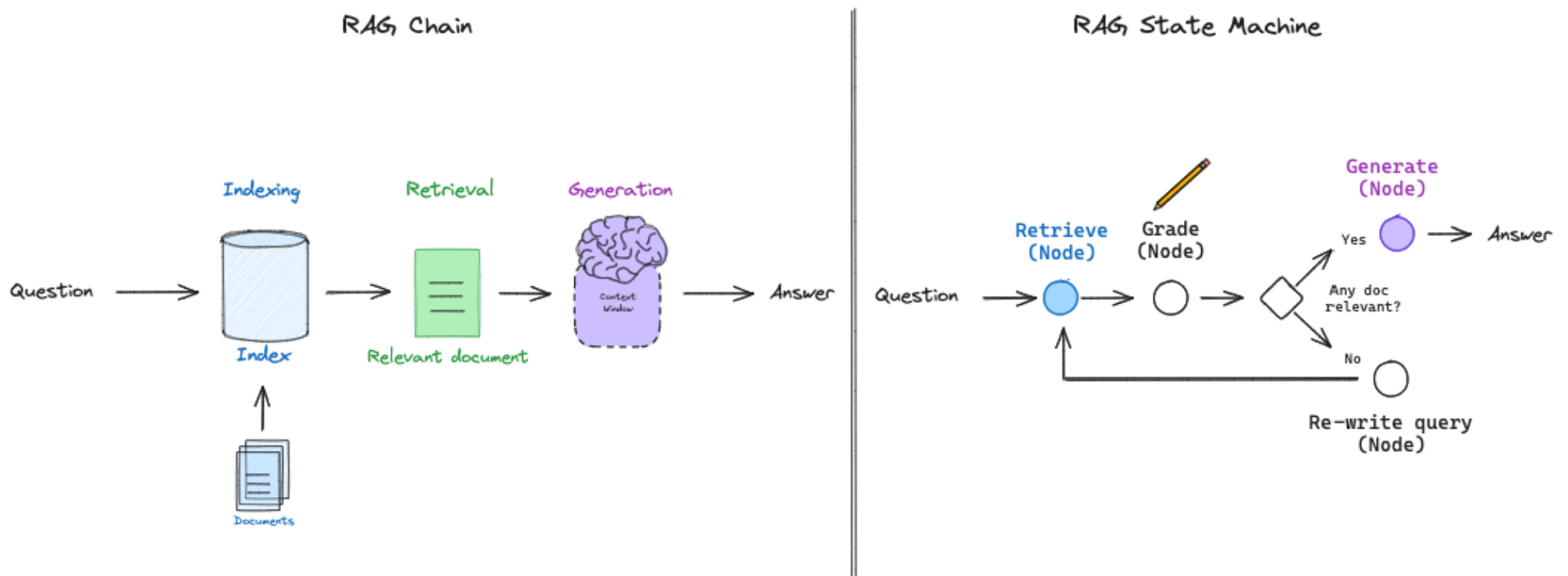


RAG with LangGraph

A Comprehensive Guide to Retrieval-Augmented Generation



Bharath Reddy
Follow For More

Why LangGraph?

What is LangGraph?

LangGraph is a framework for building stateful, multi-step AI applications using graphs. Think of it as a flowchart where each box is a function and arrows show data flow.

Why Use Graph-Based Systems?

1. **Clear Flow Visualization:** See exactly how data moves through your pipeline
2. **Stateful Processing:** Each step can access and modify shared state
3. **Easy Debugging:** Know which step failed and what data it had
4. **Modularity:** Add/remove steps without rewriting everything
5. **Conditional Routing:** Different paths based on conditions (e.g., simple questions vs complex ones)

Traditional vs Graph-Based:



```
Traditional: function1() → function2() → function3()  
(Hard to modify, debug, or add conditions)
```

```
LangGraph: [Node: Retrieve] → [Node: Generate] → [END]  
(Each node is independent, state flows between them)
```

Why for RAG?

RAG needs multiple steps (retrieve, check quality, generate, verify). LangGraph makes it easy to manage this workflow, add conditional logic (like "if retrieval is poor, try different search"), and debug each step independently.

RAG (Retrieval-Augmented Generation) combines document retrieval with LLM generation. Instead of relying only on the LLM's training data, we retrieve relevant documents and use them as context for better, more accurate responses.

PIPELINE FLOW

- Ingest documents
- Chunk them into smaller pieces
- Create embeddings (vector representations)
- Store in vector database
- Retrieve relevant chunks for queries
- Generate responses using LLM + retrieved context

WHAT WE NEED

- LangChain for document processing
- LangGraph for building the workflow
- FAISS for vector storage
- OpenAI for embeddings and LLM

INSTALLATION & IMPORTS

```
# pip install langchain langgraph langchain-openai faiss-cpu pypdf

import os
from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import FAISS
from langchain.prompts import ChatPromptTemplate
from langgraph.graph import StateGraph, END
from typing import TypedDict, List
import operator
from typing_extensions import Annotated

os.environ["OPENAI_API_KEY"] = "your-api-key-here"
```


Concept:

Load documents from various sources (PDFs, text files, etc.). The loader converts raw files into Document objects that LangChain can process.

CODE

```
def load_documents(file_paths: List[str]):
    documents = []
    for file_path in file_paths:
        if file_path.endswith('.pdf'):
            loader = PyPDFLoader(file_path)
        elif file_path.endswith('.txt'):
            loader = TextLoader(file_path)
        else:
            continue
        docs = loader.load()
        documents.extend(docs)
    return documents

# Usage: documents = load_documents(["doc1.pdf", "doc2.txt"])
```

Concept:

Break large documents into smaller chunks. This is important because embeddings work better on focused content, retrieval is more precise, and chunks fit within LLM context windows.

CODE

```
def chunk_documents(documents, chunk_size=1000,
chunk_overlap=200):
    """Split documents into smaller chunks."""
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len,
        separators=["\n\n", "\n", " ", ""]
    )

    chunks = text_splitter.split_documents(documents)
    print(f"Created {len(chunks)} chunks")
    return chunks

# Example usage
chunks = chunk_documents(documents)
```

Creating Embeddings

6 / 16

Concept:

Convert text chunks into vectors (embeddings). Similar content will have similar vectors, allowing us to find relevant chunks through similarity search.

CODE

```
def create_embeddings():  
    """Initialize the embedding model."""  
    embeddings = OpenAIEmbeddings(  
        model="text-embedding-3-small"  
    )  
    return embeddings  
  
# Example: See how embeddings work  
embeddings = create_embeddings()  
sample_text = "LangGraph is a framework for building stateful  
agents"  
sample_embedding = embeddings.embed_query(sample_text)  
print(f"Embedding dimension: {len(sample_embedding)}")
```


Concept:

Store all chunk embeddings in a vector database (FAISS). This enables fast similarity search to retrieve relevant chunks.

CODE

```
def create_vector_store(chunks, embeddings):  
    """Create FAISS vector store from document chunks."""  
    vector_store = FAISS.from_documents(  
        documents=chunks,  
        embedding=embeddings  
    )  
    print("Vector store created successfully")  
    return vector_store  
  
# Create the vector store  
vector_store = create_vector_store(chunks, embeddings)  
  
# Optional: Save to disk  
vector_store.save_local("faiss_index")
```


Concept:

Before building the full pipeline, test if retrieval works correctly. Query the vector store and see what chunks it returns.

CODE

```
def test_retrieval(vector_store, query, k=3):
    """Test the retrieval mechanism."""
    relevant_docs = vector_store.similarity_search(query, k=k)

    print(f"\nQuery: {query}")
    print(f"\nFound {len(relevant_docs)} relevant chunks:\n")

    for i, doc in enumerate(relevant_docs):
        print(f"--- Chunk {i+1} ---")
        print(doc.page_content[:300])
        print(f"Source: {doc.metadata}\n")

    return relevant_docs

# Test it out
query = "What is LangGraph used for?"
test_retrieval(vector_store, query)
```

Concept:

LangGraph uses a state-based approach. We define what data flows through our pipeline at each step.

CODE

```
class RAGState(TypedDict):  
    """Define the state that flows through our RAG pipeline."""  
    question: str # User's question  
    context: Annotated[List[str], operator.add] # Retrieved documents  
    answer: str # Final answer from LLM  
  
    # This state will be passed between nodes in our graph
```

Concept:

Create a node that takes a question and retrieves relevant chunks from the vector store.

CODE

```
def retrieve_node(state: RAGState) → RAGState:
    """Retrieval node: Takes question and retrieves relevant
    context."""
    question = state["question"]

    # Retrieve top 3 most relevant chunks
    relevant_docs = vector_store.similarity_search(question, k=3)

    # Extract text content from documents
    context = [doc.page_content for doc in relevant_docs]

    print(f"Retrieved {len(context)} relevant chunks")

    return {
        "context": context,
        "question": question
    }
```


Concept:

Create a node that takes the question and retrieved context, then generates an answer using an LLM.

CODE

```
# Initialize the LLM
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

def generate_node(state: RAGState) → RAGState:
    """Generation node: Uses retrieved context to generate an answer."""
    question = state["question"]
    context = state["context"]

    # Create a prompt template
    prompt = ChatPromptTemplate.from_messages([
        ("system", """"You are a helpful assistant ... """),
        ("human", "{question}")
    ])

    context_text = "\n\n".join(context)
    chain = prompt | llm
    response = chain.invoke({"context": context_text, "question":
question})

    return {"answer": response.content}
```

Building the LangGraph Pipeline^{12 / 16}

Concept:

Connect retrieval and generation nodes into a graph.

LangGraph manages the flow from question → retrieval → generation → answer.

CODE

```
def build_rag_graph():  
    """Build the RAG pipeline using LangGraph."""  
    workflow = StateGraph(RAGState)  
  
    # Add nodes  
    workflow.add_node("retrieve", retrieve_node)  
    workflow.add_node("generate", generate_node)  
  
    # Define the flow  
    workflow.set_entry_point("retrieve")  
    workflow.add_edge("retrieve", "generate")  
    workflow.add_edge("generate", END)  
  
    # Compile the graph  
    app = workflow.compile()  
    return app  
  
# Build the pipeline  
rag_app = build_rag_graph()
```

Complete Example - Putting It All Together

CODE

```
def complete_rag_pipeline(file_paths, questions):
    documents = load_documents(file_paths)
    chunks = chunk_documents(documents)
    embeddings = create_embeddings()
    vector_store = create_vector_store(chunks, embeddings)
    rag_app = build_rag_graph()
    for question in questions:
        ask_question(question)
    return rag_app

# Usage:
# files = ["doc1.pdf", "doc2.txt"]
# questions = ["What is the main topic?", "Summarize key points"]
# rag_app = complete_rag_pipeline(files, questions)
```

PIPELINE STEPS

- Load & chunk documents
- Create embeddings & vector store
- Build RAG graph & answer questions

Concept:

Now we can ask questions and get answers based on our documents!

CODE

```
def ask_question(question: str):
    """Run the RAG pipeline with a question."""
    print(f"\n{'='*60}")
    print(f"Question: {question}")

    # Run the graph
    result = rag_app.invoke({
        "question": question,
        "context": [],
        "answer": ""
    })

    print(f"\nAnswer:\n{result['answer']}\n")
    return result

# Example usage
ask_question("What is LangGraph and how does it work?")
```

POSSIBLE IMPROVEMENTS

- **Better Chunking:** Semantic chunking - split by meaning
- **Add Reranking:** Retrieve more docs, then rerank for best matches
- **Add Chat Memory:** Keep conversation history
- **Streaming Responses:** Stream the LLM response token by token
- **Multiple Vector Stores:** Use different stores for different document types

ADVANCED FEATURES

- Add conditional routing based on question type
- Implement query rewriting for better retrieval
- Add source citation in answers
- Use hybrid search (keyword + semantic)
- Implement evaluation metrics

You've built a complete RAG pipeline with:

WHAT YOU'VE BUILT

- ✓ Document ingestion
- ✓ Intelligent chunking
- ✓ Vector embeddings
- ✓ FAISS vector database
- ✓ LangGraph orchestration
- ✓ Context-aware generation

This pipeline can now answer questions based on your documents with high accuracy!